

Android XML 原生RadioButton详解（基础+自定义）

在 Android 传统 XML 开发中，RadioButton（单选按钮）是实现「互斥选择」场景的核心控件，比如性别选择、支付方式选择等。它与 Checkbox 的核心区别在于：**一组 RadioButton 中只能有一个被选中**，这一特性由 RadioGroup 容器来保障。本文从「原生基础用法」到「深度自定义样式/布局」全面拆解，搭配完整可运行代码，新手也能快速上手。

一、核心概念：RadioButton 是什么？

RadioButton 继承自 CompoundButton（与 Checkbox、Switch 同属复合按钮体系），核心特性如下：

- **互斥性**：必须嵌套在 RadioGroup 中使用，同一 RadioGroup 内的 RadioButton 只能有一个处于选中状态
- **状态特性**：仅支持「选中（checked=true）」和「未选中（checked=false）」两种状态，默认选中状态下显示圆点标记
- **样式关联**：原生样式遵循系统主题（如 Material Design），可通过 XML 属性快速调整，也能通过 selector、自定义布局实现完全定制
- **事件监听**：可通过 RadioGroup 的 OnCheckedChangeListener 监听选中状态变化，比单独监听每个 RadioButton 更高效

💡 注意：若 RadioButton 不放入 RadioGroup，将失去互斥性，表现与 Checkbox 类似，这是开发中常见的错误场景。

二、基础用法：原生 RadioButton（3 步搞定）

原生 RadioButton 的使用核心是「RadioGroup 包裹 + 状态监听」，适合简单的单选场景，无需自定义样式。

步骤 1：在 XML 布局中搭建结构

创建 activity_main.xml，用 RadioGroup 包裹多个 RadioButton，实现互斥选择组：

Code block

```
1 <?xml version="1.0" encoding="utf-8"?>
```

```
2  <LinearLayout
3      xmlns:android="http://schemas.android.com/apk/res/android"
4      android:layout_width="match_parent"
5      android:layout_height="match_parent"
6      android:orientation="vertical"
7      android:padding="20dp">
8
9      <TextView
10         android:layout_width="wrap_content"
11         android:layout_height="wrap_content"
12         android:text="请选择性别"
13         android:textSize="18sp"
14         android:textColor="@color/black"/>
15
16
17      <RadioGroup
18         android:id="@+id/rg_gender"
19         android:layout_width="wrap_content"
20         android:layout_height="wrap_content"
21         android:orientation="vertical"
22         android:layout_marginTop="15dp">
23
24
25         <RadioButton
26             android:id="@+id/rb_male"
27             android:layout_width="wrap_content"
28             android:layout_height="wrap_content"
29             android:text="男"
30             android:textSize="16sp"
31             android:layout_marginTop="10dp"/>
32
33
34         <RadioButton
35             android:id="@+id/rb_female"
36             android:layout_width="wrap_content"
37             android:layout_height="wrap_content"
38             android:text="女"
39             android:textSize="16sp"
40             android:layout_marginTop="10dp"
41             android:checked="true"/>
42
43
44         <RadioButton
45             android:id="@+id/rb_secret"
46             android:layout_width="wrap_content"
47             android:layout_height="wrap_content"
48             android:text="保密"
```

```

49         android:textSize="16sp"
50         android:layout_marginTop="10dp"
51         android:enabled="false"/>
52
53     </RadioGroup>
54
55
56     <TextView
57         android:id="@+id/tv_result"
58         android:layout_width="wrap_content"
59         android:layout_height="wrap_content"
60         android:layout_marginTop="20dp"
61         android:text="当前选中：女"
62         android:textSize="16sp"/>
63
64 </LinearLayout>

```

步骤 2：在 Activity 中监听状态变化

在 MainActivity.kt（或 Java）中绑定 RadioGroup，通过 OnCheckedChangeListener 监听选中状态，无需单独处理每个 RadioButton：

Code block

```

1  import android.os.Bundle
2  import android.widget.RadioGroup
3  import android.widget.TextView
4  import androidx.appcompat.app.AppCompatActivity
5
6  class MainActivity : AppCompatActivity() {
7      override fun onCreate(savedInstanceState: Bundle?) {
8          super.onCreate(savedInstanceState)
9          setContentView(R.layout.activity_main)
10
11         // 1. 绑定控件
12         val rgGender = findViewById<RadioGroup>(R.id.rg_gender)
13         val tvResult = findViewById<TextView>(R.id.tv_result)
14
15         // 2. 监听 RadioGroup 选中状态变化 (核心)
16         rgGender.setOnCheckedChangeListener { group, checkedId ->
17             // checkedId 是当前选中 RadioButton 的 id
18             val selectedText = when (checkedId) {
19                 R.id.rb_male -> "男"
20                 R.id.rb_female -> "女"
21                 R.id.rb_secret -> "保密"
22                 else -> "未选择"

```

```
23         }
24         // 更新结果文本
25         tvResult.text = "当前选中: $selectedText"
26     }
27
28     // 3. 主动设置选中状态 (可选, 如从本地缓存恢复状态)
29     // rgGender.check(R.id.rb_male)
30 }
31 }
```

步骤 3：常用 XML 属性说明

原生 RadioButton 和 RadioGroup 支持通过 XML 属性快速调整样式和行为，重点属性如下：

1. RadioButton 核心属性

属性名	作用	示例值
android:text	单选按钮旁的文本	"男"
android:checked	默认是否选中	true/false
android:enabled	是否启用（禁用后灰显，无法点击）	true/false
android:textSize	文本大小	"16sp"
android:textColor	文本颜色	"@color/blue"
android:button	自定义选中标记（替换默认圆点）	"@drawable/rb_selector"
app:buttonTint	选中标记颜色（Material 主题）	"@color/red"

2. RadioGroup 核心属性

属性名	作用	示例值
android:orientation	子 RadioButton 排列方向（垂直/水平）	vertical/horizontal
android:checkedButton	默认选中的 RadioButton id（与 RadioButton 的 checked 二选一）	"@id/rb_female"

android:layout_margin	组的外边距	"15dp"
-----------------------	-------	--------

三、进阶：自定义 RadioButton 样式（替换选中标记）

原生 RadioButton 的选中标记是固定圆点，若需修改为图标、方形或自定义颜色，核心方案是「用 selector 选择器定义不同状态的标记样式」，再通过 RadioButton 的 android:button 属性关联。

核心原理

RadioButton 的选中标记由「button drawable」控制，通过 selector 可根据 checked 状态（选中/未选中/禁用）显示不同的 drawable 资源（shape 图形或图片），实现样式自定义。

场景 1：自定义选中标记（图标+颜色）

需求：未选中显示灰色空心圆，选中显示蓝色实心圆+白色对勾，禁用时显示浅灰色。

步骤 1：创建 selector 资源文件

在 res/drawable 目录下新建 rb_selector.xml（选中标记状态选择器）：

Code block

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <selector xmlns:android="http://schemas.android.com/apk/res/android">
3
4      <item android:state_checked="true" android:state_enabled="true">
5          <layer-list>
6
7              <item>
8                  <shape android:shape="oval">
9                      <solid android:color="@color/blue"/>
10                     <size android:width="20dp" android:height="20dp"/>
11                 </shape>
12             </item>
13
14             <item
15                 android:drawable="@drawable/ic_check"
16                 android:gravity="center"
17                 android:width="12dp"
18                 android:height="12dp"/>
19         </layer-list>
20     </item>
21
22
23     <item android:state_checked="false" android:state_enabled="true">
24         <shape android:shape="oval">
```

```

25         <stroke android:width="2dp" android:color="@color/gray"/>
26         <size android:width="20dp" android:height="20dp"/>
27     </shape>
28 </item>
29
30
31 <item android:state_checked="true" android:state_enabled="false">
32     <shape android:shape="oval">
33         <solid android:color="@color/gray_light"/>
34         <size android:width="20dp" android:height="20dp"/>
35     </shape>
36 </item>
37
38
39 <item android:state_checked="false" android:state_enabled="false">
40     <shape android:shape="oval">
41         <stroke android:width="2dp" android:color="@color/gray_light"/>
42         <size android:width="20dp" android:height="20dp"/>
43     </shape>
44 </item>
45 </selector>

```

步骤 2：准备资源（颜色+图标）

1. 在 res/color/color.xml 中添加颜色资源：

Code block

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <resources>
3      <color name="blue">#2196F3</color>
4      <color name="gray">#9E9E9E</color>
5      <color name="gray_light">#E0E0E0</color>
6      <color name="black">#000000</color>
7  </resources>

```

2. 准备对勾图标 ic_check.png，放入 res/drawable 目录（可从 Material Icons 下载，或用 shape 自制）。

步骤 3：在 XML 中引用自定义样式

修改 RadioButton 的 android:button 属性，关联 selector 资源：

Code block

```

1  <RadioButton

```

```

2      android:id="@+id/rb_male"
3      android:layout_width="wrap_content"
4      android:layout_height="wrap_content"
5      android:text="男"
6      android:textSize="16sp"
7      android:layout_marginTop="10dp"
8      android:button="@drawable/rb_selector" />
9
10     <RadioButton
11         android:id="@+id/rb_female"
12         android:layout_width="wrap_content"
13         android:layout_height="wrap_content"
14         android:text="女"
15         android:textSize="16sp"
16         android:layout_marginTop="10dp"
17         android:checked="true"
18         android:button="@drawable/rb_selector" />

```

场景 2：用图片替换选中标记

若需更生动的样式（如卡通图标、品牌标识），可直接用图片作为不同状态的标记资源。

步骤 1：准备图片资源

需要 4 张图片（对应不同状态）：

- rb_checked.png：选中且启用状态
- rb_unchecked.png：未选中且启用状态
- rb_checked_disable.png：选中且禁用状态
- rb_unchecked_disable.png：未选中且禁用状态

将图片放入 res/drawable 目录，建议适配多分辨率（mdpi/hdpi/xhdpi 等）。

步骤 2：创建图片 selector

在 res/drawable 下新建 rb_image_selector.xml：

Code block

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <selector xmlns:android="http://schemas.android.com/apk/res/android">
3      <item android:state_checked="true" android:state_enabled="true"
4          android:drawable="@drawable/rb_checked"/>
5      <item android:state_checked="false" android:state_enabled="true"
6          android:drawable="@drawable/rb_unchecked"/>
7      <item android:state_checked="true" android:state_enabled="false"
8          android:drawable="@drawable/rb_checked_disable"/>
9      <item android:state_checked="false" android:state_enabled="false"
10         android:drawable="@drawable/rb_unchecked_disable"/>
11 </selector>

```

```
5      <item android:state_checked="true" android:state_enabled="false"
    android:drawable="@drawable/rb_checked_disable"/>
6      <item android:state_checked="false" android:state_enabled="false"
    android:drawable="@drawable/rb_unchecked_disable"/>
7  </selector>
```

步骤 3：引用图片样式

Code block

```
1  <RadioButton
2      android:id="@+id/rb_pay_alipay"
3      android:layout_width="wrap_content"
4      android:layout_height="wrap_content"
5      android:text="支付宝支付"
6      android:button="@drawable/rb_image_selector"
7      android:layout_marginTop="10dp"/>
```

四、高级：完全自定义 RadioButton 布局（复杂结构）

若需实现「图标+文本+描述信息+单选标记」的复杂布局（如商品规格选择），需通过「自定义布局」实现——用 LinearLayout/RelativeLayout 包裹 RadioButton 和其他控件，再通过 RadioGroup 保障互斥性。

需求

- 布局结构：商品图片 + 商品名称 + 价格 + 单选标记（靠右）
- 同一 RadioGroup 内的商品只能选中一个，选中时背景变色
- 点击整个商品项触发选中状态切换

步骤 1：创建自定义商品项布局

在 res/layout 目录下新建 layout_rb_goods.xml：

Code block

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout
3      xmlns:android="http://schemas.android.com/apk/res/android"
4      android:layout_width="match_parent"
5      android:layout_height="100dp"
6      android:orientation="horizontal"
7      android:padding="10dp"
8      android:clickable="true"
```



```
9      android:focusable="true"
10      android:background="@drawable/goods_bg_selector"
11      android:gravity="center_vertical">
12
13
14      <ImageView
15          android:id="@+id/iv_goods"
16          android:layout_width="80dp"
17          android:layout_height="80dp"
18          android:src="@drawable/goods1"
19          android:scaleType="centerCrop"/>
20
21
22      <LinearLayout
23          android:layout_width="0dp"
24          android:layout_height="wrap_content"
25          android:layout_weight="1"
26          android:orientation="vertical"
27          android:layout_marginStart="15dp">
28
29          <TextView
30              android:id="@+id/tv_goods_name"
31              android:layout_width="wrap_content"
32              android:layout_height="wrap_content"
33              android:text="纯棉T恤（白色）"
34              android:textSize="16sp"
35              android:textColor="@color/black"/>
36
37          <TextView
38              android:id="@+id/tv_goods_price"
39              android:layout_width="wrap_content"
40              android:layout_height="wrap_content"
41              android:text="¥99.00"
42              android:textSize="14sp"
43              android:textColor="@color/red"
44              android:layout_marginTop="5dp"/>
45
46      </LinearLayout>
47
48
49      <RadioButton
50          android:id="@+id/rb_goods"
51          android:layout_width="wrap_content"
52          android:layout_height="wrap_content"
53          android:button="@drawable/rb_selector"
54          android:text="" />
55
```

补充：商品项背景选择器（goods_bg_selector.xml）

实现选中时背景变色的效果，在 res/drawable 下创建：

Code block

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <selector xmlns:android="http://schemas.android.com/apk/res/android">
3
4      <item android:state_checked="true"
5          android:drawable="@color/blue_light_transparent"/>
6
7      <item android:drawable="@android:color/transparent"/>
8  </selector>
```

在 color.xml 中添加半透明颜色：

Code block

```
1  <color name="blue_light_transparent">#1A2196F3</color>
2  <color name="red">#F44336</color>
```

步骤 2：在主布局中用 RadioGroup 包裹自定义项

修改 activity_main.xml，用 RadioGroup 承载多个自定义商品项：

Code block

```
1  <TextView
2      android:layout_width="wrap_content"
3      android:layout_height="wrap_content"
4      android:text="请选择商品"
5      android:textSize="18sp"
6      android:textColor="@color/black"
7      android:layout_marginTop="30dp"/>
8
9  <RadioGroup
10     android:id="@+id/rg_goods"
11     android:layout_width="match_parent"
12     android:layout_height="wrap_content"
13     android:orientation="vertical"
14     android:layout_marginTop="15dp">
15
```

```
16
17     <include layout="@layout/layout_rb_goods" android:id="@+id/item_goods1"/>
18
19     <include layout="@layout/layout_rb_goods" android:id="@+id/item_goods2"/>
20
21     <include layout="@layout/layout_rb_goods" android:id="@+id/item_goods3"/>
22
23 </RadioGroup>
```

步骤 3：在 Activity 中初始化并控制状态

在 MainActivity.kt 中初始化商品项数据，监听 RadioGroup 选中状态：

Code block

```
1  import android.os.Bundle
2  import android.widget.ImageView
3  import android.widget.RadioButton
4  import android.widget.RadioGroup
5  import android.widget.TextView
6  import androidx.appcompat.app.AppCompatActivity
7
8  class MainActivity : AppCompatActivity() {
9      // 商品数据（模拟）
10     private data class Goods(val id: Int, val name: String, val price: String,
11     val imgRes: Int)
12     private val goodsList = listOf(
13         Goods(1, "纯棉T恤（白色）", "¥99.00", R.drawable.goods1),
14         Goods(2, "牛仔裤（蓝色）", "¥159.00", R.drawable.goods2),
15         Goods(3, "运动鞋（黑色）", "¥299.00", R.drawable.goods3)
16     )
17
18     override fun onCreate(savedInstanceState: Bundle?) {
19         super.onCreate(savedInstanceState)
20         setContentView(R.layout.activity_main)
21
22         val rgGoods = findViewById<RadioGroup>(R.id.rg_goods)
23         // 初始化商品项
24         initGoodsItems(rgGoods)
25
26         // 监听商品选中状态
27         rgGoods.setOnCheckedChangeListener { _, checkedId ->
28             val selectedGoods = when (checkedId) {
29                 R.id.item_goods1 -> goodsList[0]
30                 R.id.item_goods2 -> goodsList[1]
31                 R.id.item_goods3 -> goodsList[2]
32                 else -> null
33             }
```

```
32         }
33         selectedGoods?.let {
34             println("选中商品: ${it.name}, 价格: ${it.price}")
35             // 可在此处更新UI或提交数据
36         }
37     }
38 }
39
40 // 初始化商品项数据
41 private fun initGoodsItems(rgGoods: RadioGroup) {
42     for (i in goodsList.indices) {
43         val goods = goodsList[i]
44         // 获取include的商品项
45         val itemView = rgGoods.getChildAt(i)
46         // 绑定控件
47         val ivGoods = itemView.findViewById<ImageView>(R.id.iv_goods)
48         val tvName = itemView.findViewById<TextView>(R.id.tv_goods_name)
49         val tvPrice = itemView.findViewById<TextView>(R.id.tv_goods_price)
50         val rbGoods = itemView.findViewById<RadioButton>(R.id.rb_goods)
51
52         // 设置数据
53         ivGoods.setImageResource(goods.imgRes)
54         tvName.text = goods.name
55         tvPrice.text = goods.price
56
57         // 点击商品项时, 选中对应的RadioButton
58         itemView.setOnClickListener {
59             rbGoods.isChecked = true
60         }
61     }
62 }
63 }
```

五、关键知识点总结（必记）

知识点	核心内容
互斥性保障	RadioButton 必须嵌套在 RadioGroup 中，否则失去互斥性
状态监听	优先通过 RadioGroup 的 OnCheckedChangeListener 监听，无需单独处理每个 RadioButton

样式自定义核心	通过 RadioButton 的 android:button 属性关联 selector，控制选中标记的状态样式
布局自定义思路	用 LinearLayout 包裹 RadioButton 和其他控件，外层用 RadioGroup 保障互斥，扩大点击范围
默认选中设置	两种方式：RadioButton 的 android:checked="true" 或 RadioGroup 的 android:checkedButton="@id/xxx"

六、常见问题排查

1. RadioButton 不互斥

原因：未将 RadioButton 放入同一个 RadioGroup 中；或多个 RadioGroup 嵌套导致分组错误。

解决：确保需要互斥的 RadioButton 属于同一个 RadioGroup，避免 RadioGroup 嵌套。

2. 自定义样式不生效

原因：selector 中状态顺序错误（具体状态需放在前面）；drawable 资源文件名含大写或特殊字符；RadioButton 未关联 android:button 属性。

解决：selector 按「选中启用→未选中启用→选中禁用→未选中禁用」排序；资源文件名仅用小写+下划线；确认 android:button 引用正确。

3. 点击商品项不触发选中

原因：商品项布局未设置 clickable="true"；RadioButton 的 isChecked 未在点击事件中设置。

解决：给外层布局添加 clickable="true" 和 focusable="true"；在布局点击事件中调用 rbGoods.isChecked = true。

4. Material 主题属性（app:xxx）不生效

原因：未依赖 Material Design 库；布局根节点未添加 app 命名空间。

解决：在 build.gradle 中添加依赖 `implementation 'com.google.android.material:material:1.12.0'`；布局根节点添加 `xmlns:app="http://schemas.android.com/apk/res-auto"`。

通过以上内容，你可以覆盖 RadioButton 从基础到复杂的全场景使用需求。如果需要实现更特殊的效果（如单选按钮组横向滚动、带动画的选中效果），可以随时补充需求！

（注：文档部分内容可能由 AI 生成）